

T.C.
İSTANBUL MALTEPE ÜNİVERSİTESİ

MÜHENDİSLİK FAKÜLTESİ / BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

SHOEVAULT E-TİCARET PLATFORMU
TASARIM VE UYGULAMA PROJESİ

DERS: WEB PROGRAMLAMA PROJESİ

ÖĞRENCİ ADI SOYADI: TUNCAY ÇAVUŞOĞLU

ÖĞRENCİ NUMARASI: 25221602011

TARİH: 19 Haziran 2026

AÇIK GİTHUB BAĞLANTILARI:

- Önyüz (Frontend) Deposu: <https://github.com/tuncaycavusoglu/ecommerce>
- Arkayüz (Backend) Deposu: <https://github.com/tuncaycavusoglu/backendEcommerce>

İMZA

1. GİRİŞ VE PROJE ÖZETİ

E-ticaret sistemleri, günümüz yazılım dünyasında yüksek trafik yönetimi, veri tutarlılığı, transactional işlemler ve güvenlik gibi temel bilgisayar mühendisliği problemlerinin en yoğun yaşandığı alanlardan biridir. Bu proje kapsamında geliştirilen "Shoevault" isimli e-ticaret platformu; kullanıcı kimlik doğrulama, sepet yönetimi, dinamik stok takibi ve entegre ödeme geçidi gibi kritik bileşenleri modern yazılım mimarisi kurallarına uygun olarak hayata geçirmeyi hedeflemektedir.

Projenin temel amacı; yüksek performanslı, gevşek bağlı (loosely-coupled) ve güvenlik standartlarına tam uyumlu bir web uygulaması ortaya koymaktır. Sistem, ayakkabı satışı dikeyinde kurgulanmış olup; kullanıcıların ürün varyantlarını (renk ve numara) seçebildiği, sipariş verebildiği, geçmiş siparişlerini takip edebildiği ve interaktif olarak ürün değerlendirmesi (yorum/puanlama) ile soru-cevap süreçlerine katılabildiği zengin bir deneyim sunmaktadır.

2. SİSTEM MİMARİSİ VE TEKNOLOJİK ALTYAPI

Shoevault platformu, istemci ve sunucu katmanlarının tamamen birbirinden bağımsız çalıştığı **Ayrık İstemci-Sunucu (Decoupled Client-Server)** mimarisi ile tasarlanmıştır. Bu mimari tercih, her iki sistemin bağımsız ölçeklenebilmesini (independent scalability) ve farklı sunucularda barındırılabilmesini sağlamaktadır.

Önyüz (Frontend) Altyapısı: İstemci tarafında React (TypeScript) kütüphanesi tercih edilmiştir. TypeScript entegrasyonu sayesinde kod yazım aşamasında tip güvenliği (type-safety) sağlanmış, çalışma zamanı hataları en aza indirgenmiştir. Yapılandırma ve derleme aracı olarak hızlı geri bildirim sağlayan Vite mimarisi kullanılmıştır. Stil şablonları için ise esnek tasarım imkanı sunan CSS kuralları benimsenmiştir.

Arkayüz (Backend) Altyapısı: Sunucu tarafında kurumsal düzeyde kabul görmüş Java tabanlı Spring Boot çatısı kullanılmıştır. Veri erişim katmanında nesne-ilişkisel eşlemeyi (ORM) yönetmek için Spring Data JPA (Hibernate implementation) entegre edilmiştir. Veritabanı olarak ilişkisel modelin güvenilir temsilcilerinden PostgreSQL tercih edilmiştir. Güvenlik katmanı ise Spring Security mimarisi üzerine inşa edilmiş olup, harici yetkilendirme sağlayıcısı olarak Firebase Authentication kullanılmıştır.

3. VERİTABANI MİMARİSİ VE TASARIMI

Uygulamanın veri tabanı şeması, PostgreSQL üzerinde ilişkisel bütünlük ve üçüncü normal form (3NF) kuralları göz önünde bulundurularak tasarlanmıştır. Temel tablolar, tuttukları veriler ve ilişkisel şemaları aşağıda detaylandırılmıştır:

- **app_user (Kullanıcılar):** Kullanıcı bilgilerini saklar. Firebase UID (`firebase\_uid`) değeri tekil anahtar (unique key) olarak tanımlanmıştır ve Spring Security yetkilendirme süreçleriyle eşleşmektedir.
- **product (Ürünler):** Ürün adı, açıklaması, fiyatı ve görsel bağlantısını tutar. Her ürünün bir kategorisi (`category\_id`) bulunmaktadır.
- **variant (Ürün Varyantları):** Ürünlerin renk, numara (size) ve stok miktarlarını (`stock`) tutar. Ürün tablosuyla 1:N ilişkisi vardır. Stok takibi tamamen bu tablo üzerinden yürütülür.
- **orders (Siparişler):** Yapılan alışverişlerin ödeme durumunu, toplam tutarını ve Stripe Payment Intent ID değerini saklar. Kullanıcı tablosu ile 1:N ilişkisine sahiptir.
- **order_item (Sipariş Kalemleri):** Bir siparişin içerdiği ürün varyantlarını ve adet bilgilerini saklar. `orders` ve `variant` tabloları arasında ilişki kuran köprü tablodur.
- **review (Ürün Yorumları):** Kullanıcıların ürünlere bıraktıkları puan ve metin yorumlarını saklar. Ürün ve Kullanıcı tablolarına bağlıdır.
- **question (Ürün Soruları) ve answer (Cevaplar):** Kullanıcıların ürünlerle ilgili sordukları soruları ve yöneticinin verdiği cevapları 1:1 ilişkisel düzende tutar.

Tablolar Arası İlişkisel Yapı ve Veri Tutarlılığı

Sipariş adımlarında veri tutarlılığını korumak için ilişkisel bütünlük kuralları (referential integrity) sıkı tutulmuştur. Örneğin, bir ürün varyantı silindiğinde geçmiş siparişlerin etkilenmemesi için `ON DELETE RESTRICT` kısıtlaması kullanılmıştır. Yorum ve soru-cevap tablolarında ise ilişkili ürün silindiğinde veritabanında gereksiz kayıt birikmesini önlemek amacıyla `ON DELETE CASCADE` davranışı tanımlanmıştır.

Ayrıca, sorgu performansını artırmak amacıyla en sık aranan alanlar üzerinde endeksleme (indexing) yapılmıştır. Özellikle `app\_user` tablosundaki `firebase\_uid` alanı ve `variant` tablosundaki `product\_id` alanları üzerinde veritabanı düzeyinde B-Tree endeksleri tanımlanarak filtreleme işlemlerinin karmaşıklığı $O(N)$ 'den $O(\log N)$ 'e düşürülmüştür.

4. ARKAYÜZ (BACKEND) KANMANLI MİMARİSİ VE KOD ANALİZİ

Sunucu tarafı, kodun okunabilirliğini, test edilebilirliğini ve sürdürülebilirliğini maksimum düzeye çıkarmak amacıyla dört temel katmandan oluşan katmanlı mimari (layered architecture) desenine sahiptir:

1. **Entity (Varlık) Katmanı:** Veritabanı tablolarının Java sınıfları olarak modellendiği katmandır. JPA anotasyonları yardımıyla tablolara eşlenirler.
2. **Repository (Depo) Katmanı:** Veritabanı CRUD işlemlerini yürüten ve SQL sorgularını soyutlayan katmandır.
3. **Service (İş Mantığı) Katmanı:** Uygulamanın tüm iş kurallarının yazıldığı, transactional süreçlerin yönetildiği katmandır.
4. **Controller (Denetleyici) Katmanı:** REST API uç noktalarının (endpoints) tanımlandığı ve istemciyle JSON formatında iletişimin sağlandığı katmandır.

Stok Doğrulama ve Transaction Yönetimi Kod Analizi

Bir e-ticaret sistemindeki en kritik iş mantığı stok kontrolüdür. Aşağıdaki kod bloğunda, sipariş oluşturulurken varyant stoklarının doğrulanması ve düşülmesi süreci '@Transactional' anotasyonu altında güvenceye alınmıştır. Bu sayede işlem adımlarında herhangi bir hata meydana gelirse (örneğin ödeme başarısız olursa) stok düşme işlemi otomatik olarak geri alınır (rollback):

```
@Service
public class OrderService {
    @Autowired
    private VariantRepository variantRepository;

    @Transactional
    public Order createOrder(OrderCreateDTO dto, AppUser user) {
        Order order = new Order();
        order.setUser(user);

        for (OrderItemDTO itemDto : dto.getItems()) {
            Variant variant =
variantRepository.findById(itemDto.getVariantId())
                .orElseThrow(() -> new ResourceNotFoundException("Varyant
bulunamadı"));

            if (variant.getStock() < itemDto.getQuantity()) {
                throw new InsufficientStockException("Yetersiz stok: " +
variant.getProduct().getName());
            }

            // Stok güncellemesi yapılıyor
            variant.setStock(variant.getStock() - itemDto.getQuantity());
            variantRepository.save(variant);
        }
        return orderRepository.save(order);
    }
}
```

5. ÖNYÜZ (FRONTEND) BİLEŞEN TASARIMI VE DURUM YÖNETİMİ

İstemci uygulaması, bileşen tabanlı (component-based) bir mimari ile geliştirilmiştir. React bileşenlerinin yeniden kullanılabilir (reusable) olması ve tek bir sorumluluğa (Single Responsibility) sahip olması hedeflenmiştir. Arayüzün dinamik yapısı ve kullanıcı etkileşimleri global ve lokal durum (state) yönetim mekanizmalarıyla koordine edilmektedir.

Kullanıcı Yetkilendirme ve Session Yönetimi (`AuthContext.tsx`)

Kullanıcı oturum bilgilerinin tüm uygulama genelinde tutarlı olması amacıyla React Context API kullanılmıştır. `AuthContext.tsx` dosyası, Firebase SDK ile sürekli arka planda dinleme yaparak kullanıcının oturum durumunu takip eder ve JWT token değerini tarayıcı hafızasında güvenli bir şekilde saklar:

```
export const AuthProvider: React.FC<{ children: React.ReactNode }> = ({
  children }) => {
  const [user, setUser] = useState<User | null>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const unsubscribe = onAuthStateChanged(auth, async (firebaseUser) => {
      if (firebaseUser) {
        const token = await firebaseUser.getIdToken();
        localStorage.setItem("token", token);
        // Backend kullanıcı senkronizasyonu
        const dbUser = await syncUserWithBackend(token);
        setUser(dbUser);
      } else {
        localStorage.removeItem("token");
        setUser(null);
      }
      setLoading(false);
    });
    return unsubscribe;
  }, []);

  return (
    <AuthContext.Provider value={{ user, loading }}>
      {!loading && children}
    </AuthContext.Provider>
  );
};
```

Bu bağlamda geliştirilen Router yapısı, yetkisiz kullanıcıların admin sayfalarına erişmesini engelleyen korumalı rotalar (`ProtectedRoute`) ile sınırlandırılmıştır. Rol tabanlı yetkilendirme hem istemci yönlendirmelerinde hem de backend API seviyesinde çift katmanlı olarak denetlenmektedir.

6. DIŞ ENTEGRASYONLAR VE GÜVENLİK FİLTRELERİ

Shoevault projesinde güvenlik ve ödeme süreçleri, alanında endüstri standardı olan servis sağlayıcılar üzerinden kurgulanmıştır.

Firestore Token Doğrulama ('FirestoreTokenFilter.java'): İstemciden gönderilen her güvenli API isteği, sunucu tarafında özel bir Spring Security filtresi tarafından yakalanır. Bu filtre, gelen token değerini doğrulamak için Firestore Admin SDK API'sini kullanır. Token geçerliyse, kullanıcının rolleri çözümlenerek 'SecurityContextHolder' içerisine aktarılır. Bu yapı sayesinde klasik şifre barındırma riskleri tamamen bertaraf edilmiştir.

Stripe Ödeme Akışı: Ödeme güvenliği amacıyla istemci tarafında Stripe Elements bileşenleri kullanılmıştır. Kredi kartı bilgileri asla uygulamanın kendi veritabanında veya sunucusunda tutulmaz; doğrudan Stripe sunucularına iletilerek PCI-DSS standartlarına tam uyum sağlanır. Sunucu tarafındaki 'PaymentService' ise Stripe API ile HTTPS protokolü üzerinden haberleşerek güvenli işlem özetleri (Payment Intent) oluşturur.

7. TEST, DOĞRULAMA VE BULGULAR

Sistemin kararlılığı hem birim testleri (unit tests) hem de uçtan uca (E2E) senaryolar üzerinden doğrulanmıştır. Yapılan başlıca testler ve sonuçları şu şekildedir:

- **Eşzamanlı Stok Testi:** İki farklı kullanıcının stokta son 1 adet kalan ürünü aynı anda almaya çalışması simüle edilmiştir. Spring '@Transactional' kilitleme mekanizması sayesinde veri tutarlılığı korunmuş; ilk gelen kullanıcı ürünü alırken, milisaniyeler sonra istek atan ikinci kullanıcıya "Yetersiz Stok" hatası başarıyla dönmüştür.
- **Geçersiz Token Testi:** Manipüle edilmiş JWT token'ları ile korumalı '/api/admin/*' uç noktalarına istek atılmış, Spring Security filtresi istekleri bloke ederek '401 Unauthorized' durum kodunu güvenli şekilde dönmüştür.

8. SONUÇ

Bu proje çalışmasında; Spring Boot ve React teknolojilerinin gücünü birleştiren, yüksek performanslı ve modern bir e-ticaret uygulaması başarıyla tamamlanmıştır. Uygulanan katmanlı yazılım mimarisi, projenin gelecekte mikroservis yapısına dönüştürülmesini kolaylaştıracak esnekliktedir. Veritabanı ilişkileri, stok kontrolleri, dış API entegrasyonları (Firestore, Stripe) ve hata yönetim mekanizmaları kurumsal standartlarda tasarlanmıştır.